

Energy Analyst Programming Exercise – Analytical Tasks

Use of AI Tools: AI tools are **permitted and encouraged** for this exercise. You may use any AI assistant to help you write or debug code, explore data, and generate visualizations. Your evaluation will include the correctness and quality of your outputs and your demonstrated understanding of the analytical problems — not whether the code was written by hand.

Before beginning, create a clean new directory that will hold all the output you produce (CSV files, images, etc.).

Task 1: Read in all the historical data files from the *historicalPriceData* folder and combine them into a single data structure (data frame, table, etc.).

Locational Nature of Power Prices

Power prices vary by location on the grid due to supply/transmission availability, regional demand, and many other factors.

Task 2: Compute the average price for each settlement point and year-month in the historical dataset (48 year-months: January 2016 through December 2019).

Compute monthly average prices for **all** settlement points (both hubs and load zones). Hubs are denoted by the prefix **HB_** and load zones by the prefix **LZ_** in the SettlementPoint name.

Do **not** filter out prices less than or equal to zero when computing averages. Negative prices are possible in deregulated power markets.

Task 3: Write the computed monthly average prices to a CSV named *AveragePriceByMonth.csv*.

The file should have four columns: SettlementPoint, Year, Month, AveragePrice.

Price Volatility

Hourly volatility is defined as the standard deviation in log returns of hourly prices.

Task 4: Compute the hourly price volatility for each year and each settlement hub (prefix **HB_**) in the historical data.

Filter out prices that are zero or negative before computing log returns (the natural logarithm is only defined for positive values).

Do **not** compute volatilities for load zones (prefix **LZ_**).



Task 5: Write the computed hourly volatilities to a CSV named `HourlyVolatilityByYear.csv`.

The file should have three columns: `SettlementPoint`, `Year`, `HourlyVolatility`. Column names are case-sensitive.

Task 6: Determine which settlement hub had the highest hourly volatility for each historical year. Extract those rows and write them to a CSV named `MaxVolatilityByYear.csv`.

Your file should contain the same column names as the file you wrote in Task 5.

Data Translation and Formatting

cQuant's price simulation models consume hourly historical spot price data as input. Each price variable represents a distinct settlement point on the grid and must be submitted as a separate CSV file in a specific format. Examples of correctly formatted files are provided in the *supplementalMaterials* folder.

Task 7: Using the files in *supplementalMaterials* as format examples, translate the data structure from Task 1 into the cQuant model-ready format and write a separate file for each settlement point.

The historical data uses the **hour-beginning** convention.

Each CSV must contain data for **one and only one** `SettlementPoint`. There should be **15 files** in total; one for each of the settlement points represented in the historical power price data.

Save these files in a subdirectory named **formattedSpotHistory** within your main output directory.

Use the filename convention: `spot_<SettlementPointName>.csv`.

Bonus – Mean Plots

Generate two line plots that display the monthly average prices you computed in Task 2 in chronological order, according to the following specification:

The first plot should show the monthly average prices for all settlement hubs only.

The second plot should show the monthly average prices for all load zones only.

In both plots, all relevant settlement points of a given type should be overlaid on the plot as separate curves, with a different color assigned to each curve. The plot should include a legend that allows the user to decode which color maps to which settlement point.



Save both plots to file as either PNG or JPEG image files, with the filenames `SettlementHubAveragePriceByMonth.png` and `LoadZoneAveragePriceByMonth.png` (or corresponding filenames for JPEG files).

Hint: To preserve chronological order, you should create a date column that you use for the x-axis dimension in your plots. You may assign each computed monthly average to the first day of the month to which it applies. This way, your plots will have x-axis tick labels of “2016-01-01”, “2016-02-01”, etc.

Bonus – Volatility Plots

Create a plot/plots that compare volatility across settlement hubs by year that you computed in Task 4. Save the plot(s) in your output directory. Use your judgement for how to present the data in the most visually effective manner. You may create as many plots as you feel is appropriate.

Bonus – Hourly Shape Profile Computation

An hourly shape profile is a normalized version of the 24 average hourly prices for a given price location over some period of historical data. It helps illustrate how the price varies, on average, by month of year, day of week, and hour of day.

Compute normalized hourly shape profiles from the historical spot price data for each settlement point by month of year and day of week.

Each hourly shape profile has 24 values, one for every hour of the day, and there are $12 \times 7 = 84$ hourly shape profiles for each price variable, one for each combination of month of year and day of week.

The average across the 24 hours in each hourly shape profile must be exactly equal to one.

Each SettlementPoint should have its hourly shape profiles saved in a separate file. There should be 15 files in total; one for each of the settlement points represented in the historical power price data.

Save these files in a subdirectory named **hourlyShapeProfiles** within your main output directory.

Use the filename convention: `profile_<SettlementPointName>.csv`.

Bonus – Open-Ended Analysis

If you have completed responses to all other tasks, with what time you have left, write code to analyze the data further in some way of your choosing. That is, tell us and/or show us something interesting about the data. This is a chance for you to show off.

Plots, models, machine learning, you name it: it's all in bounds. You may provide



narrative around your analysis within comments in your code; we will read the comments and the associated code for your analysis upon reviewing your response. Save any relevant output to your output folder.

Again, this question is entirely open-ended; use your intellectual curiosity and analytical prowess to guide you to some interesting data-driven insights.